

Architecture Microservices

Des composants clés à la mise en place
d'un système robuste et résilient

Par **Mouhamed Ndiaye** · Développeur Full Stack

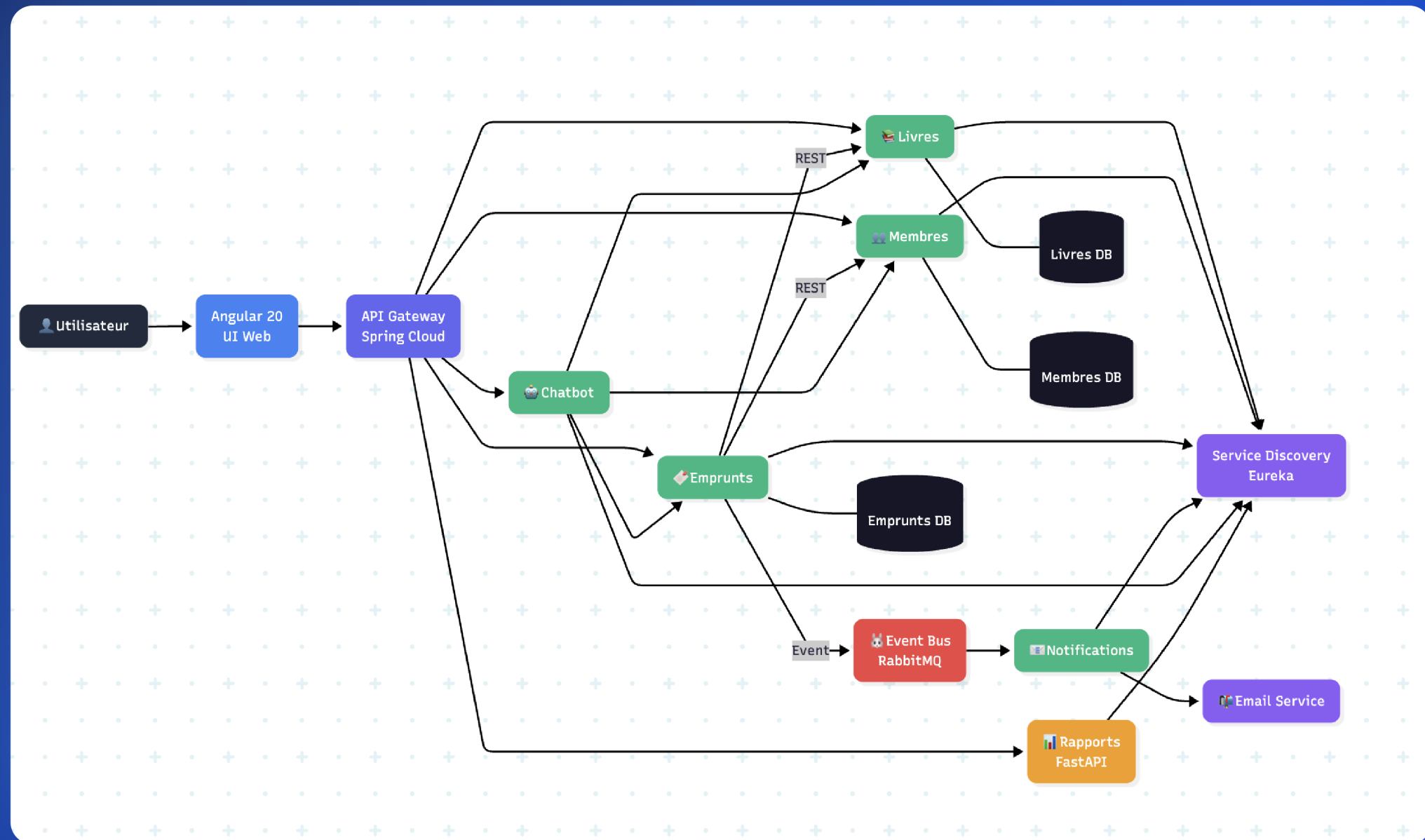
Java · Spring Boot

Python · FastAPI

Angular 20

RabbitMQ

Spring Cloud



01

Monolithe vs Microservices

Comprendre le problème avant de choisir la solution



Tout dans un seul et unique bloc

- ✗ Tout dans un seul bloc de code
- ✗ Un seul déploiement global — tout ou rien
- ✗ Scaling de toute l'application à la fois
- ✗ Une seule technologie imposée à toutes les équipes
- ✗ Code spaghetti difficile à maintenir
- ✗ Une panne = toute l'application tombe

Les Microservices

Un service indépendant par domaine métier

✓ Services indépendants par domaine métier

✓ Déploiement service par service

✓ Scaling ciblé sur un seul service

✓ Liberté totale de technologie par équipe

✓ Code maîtrisé et focalisé par service

✓ Un service tombe — les autres continuent

02

Les bénéfices clés

Pourquoi adopter cette architecture ?

Les bénéfices clés

4 raisons majeures d'adopter les microservices

03

4 Principes fondamentaux

Les règles à respecter pour une architecture saine

Les 4 principes clés

Les fondations d'une architecture microservices solide

1 Responsabilité unique — un service = un domaine métier

2 Couplage faible — chaque service évolue indépendamment

3 Haute cohésion — fonctionnalités liées dans le même service

4 Données décentralisées — une BDD dédiée par service

04

Composants indispensables

Les briques architecturales d'un système robuste

API Gateway

La porte d'entrée unique — toutes les requêtes passent par lui

 **Routage** — redirige vers le bon service selon le path

 **Authentication** — vérifie le JWT avant de laisser passer

 **Load balancing** — répartit la charge entre les instances

 **Discovery** — s'appuie sur Eureka pour trouver les services

Service Discovery — Eureka

Comment les services se trouvent mutuellement lors du scaling ?

Message Broker — RabbitMQ

Communication asynchrone — aucun couplage direct entre services



Synchrone (REST) — réponse attendue immédiatement



Asynchrone (RabbitMQ) — événement publié, consommé indépendamment



Notre flux — emprunts-svc publie → notification-svc envoie l'email

05

Notre projet

Bibliothèque Scolaire — application concrète de tous ces principes

Bibliothèque Scolaire — Architecture

8 services, 2 langages, 1 gateway, Telegram Bot



09

Démonstration

Bibliothèque Scolaire en action — 8 services, 2 langages

Eureka · services enregistrés

Gateway · routing automatique

Angular · frontend

Python · rapports-svc

Telegram · chatbot

07 Challenges à anticiper

Les microservices apportent une complexité qu'il faut maîtriser

Les défis des microservices

La puissance vient avec une responsabilité accrue



Transactions distribuées — cohérence des données entre services (SAGA)



Sécurité — authentification centralisée, secrets gérés



Observabilité — logs centralisés, traces distribuées (ELK, Jaeger)



Complexité opérationnelle — Docker, Kubernetes, CI/CD

08

En résumé

Ce qu'il faut retenir

Ce qu'il faut retenir

L'architecture microservices est un parcours, pas une destination

✓ **Services indépendants — un domaine = un service avec sa propre BDD**

✓ **Scalabilité ciblée — on scale uniquement ce qui en a besoin**

✓ **Liberté technologique — Java, Python, Node.js selon le besoin**

✓ **Résilience — un service en panne n'impacte pas les autres**

✓ **Communication claire — REST synchrone ou RabbitMQ asynchrone**

MERCI

Des questions ?

Bibliothèque Solaire — Architecture Microservices

8 services

2 langages

1 frontend

1 message broker

